

QF-AgentOS

QF-AgentOS — An honest benchmark and evidence harness for quantum finance

QF-AgentOS contributors

Version 0.1.0 — 2026-07-16

Contents

The headline result	1
Install	2
Quickstart	2
Problem families	2
What actually runs	2
Real quantum hardware	3
Research-grade extras	3
Surfaces	3
The agent team	3
Human-control model (autonomy levels)	4
What this can and cannot tell you	4
Configuration	4
Project layout	4
Documentation	5
Disclaimer	5
License	5

An honest benchmark and evidence harness for quantum finance.

Give QF-AgentOS a financial problem. A team of deterministic agents formulates it, builds **strong** classical baselines, identifies a quantum-compatible sub-problem, runs it (simulator or real QPU), **mechanically re-verifies** the result against the exact constraints, and reports — with regulatory-grade evidence — whether quantum actually helped.

The defining principle: *an agent that knows **when not** to use quantum computing is more valuable than one that always produces a circuit.*

Most quantum-finance demos are rigged: weak classical baselines, cherry-picked instances, encoding losses swept under the rug, “we ran it on hardware” hiding heavy classical post-processing. This is the opposite. **It is built so it cannot lie to you — including about quantum losing.**

The headline result

Across **5 problem families**, **3 quantum techniques**, and **real IBM hardware**, with fair comparators and mechanical verification:

\$ qf-agent arena

7 problems · 0 quantum advantage · 1 parity · 6 classical preferred.

Family	Task	Classical	Quantum	Honest verdict
collateral_allocation	optimization	MILP 7,140	QAOA — <i>(infeasible)</i>	classical preferred
payment_routing	optimization	MILP 1,869	QAOA — <i>(infeasible)</i>	classical preferred
settlement_netting	optimization	MILP 0	QAOA 0	quantum parity

Family	Task	Classical	Quantum	Honest verdict
fraud_detection	classification	AUC 0.983	quantum kernel AUC 0.557	classical preferred
rfq_fill	classification	AUC 0.945	quantum kernel AUC 0.496	classical preferred

Quantum won zero times. The QAOA solutions that *did* decode were infeasible against the exact constraints; the quantum kernels performed at or below chance (0.496 is worse than a coin flip). On real IBM hardware (ibm_kingston) the circuit also decoded an *infeasible* solution — reported honestly, not hidden.

Reproduce it yourself with `qf-agent arena`. See [docs/FINDINGS.md](#) for the full evidence and — more importantly — *why* each one loses. (Spoiler: the encoding decides the outcome before the solver ever runs.)

That is not a bug. **That is the product.**

Install

```

pip install -e "[all,dev]"      # classical + qiskit + pennylane + server + tests
# minimal (classical only):      pip install -e .
# gate-model simulation:         pip install -e "[qiskit]"
# REST API + Studio:            pip install -e "[server]"
# real hardware:                pip install -e "[ibm]" / "[dwave]"
# registries:                   pip install -e "[mlflow]" / "[postgres]"

```

Requires Python ≥ 3.11 .

Quickstart

```

qf-agent solve examples/collateral-allocation.yaml # full pipeline + evidence bundle
qf-agent arena --out arena/                       # benchmark every family (the leaderboard)
qf-agent backends                                 # what's actually available right now
qf-agent estimate --qubits 4                      # amplitude estimation vs classical MC
qf-agent simulability examples/collateral-allocation.yaml # tensor-network classical check
qf-agent serve                                     # REST API + Studio on :8000

```

Full command reference: [docs/CLI.md](#).

Problem families

The pipeline is problem-agnostic (a `ProblemDomain` abstraction); a family plugs into the same agents, backends, verification, audit, and governance. Two task types — optimization (MILP + QAOA) and classification (classical baselines + quantum kernels) — are selected automatically.

- **collateral_allocation** — minimise posting cost s.t. coverage, HQLA, concentration.
- **payment_routing** — generalized assignment: route transactions at least expected cost.
- **settlement_netting** — RTGS gridlock resolution: settle a batch s.t. per-participant liquidity.
- **fraud_detection** — quantum fidelity kernel vs RBF, same learner, temporal holdout.
- **rfq_fill** — will a request-for-quote fill? (shares the quantum-kernel base).

Details + encoding losses: [docs/PROBLEM-FAMILIES.md](#).

What actually runs

For every optimisation problem the pipeline compares *like with like*:

Layer	Method	Backend
Full problem — lower bound	Continuous LP relaxation	SciPy/HiGHS

Layer	Method	Backend
Full problem — fair comparator & recommendation	Binary MILP (exact)	SciPy/HiGHS
Research instance — classical optimum	Binary MILP (reduced)	SciPy/HiGHS
Research instance — QUBO ground truth	Brute-force enumeration	CPU
Research instance — strong heuristic	Simulated annealing	CPU
Research instance — quantum	QUBO → Ising → QAOA	statevector sim / real QPU

The **Verification agent** re-checks every decoded quantum solution against the full exact constraint set — so relaxation artifacts are caught, never hidden.

Real quantum hardware

IBM is validated end-to-end (a clean 9-step run on `ibm_kingston`); D-Wave annealing is wired and mock-tested. Both are credential-gated behind autonomy L3 + explicit approval + budget, and fall back to the simulator with a recorded reason.

```
execution_policy:
  qpu_backend: ibm      # or: dwave   (default: sim)
  autonomy_level: L3
  max_qpu_budget_usd: 5
```

Runbook: [docs/real-qpu.md](#). Credentials are read from the environment as secrets and are never logged or written into evidence.

Research-grade extras

- **Quantum Amplitude Estimation** (`qf-agent estimate`) — MLAE with a real Grover operator vs exact summation + Monte Carlo. Verdict: the quadratic speedup is *proven but asymptotic*, $O(2^m)$ state-prep-bottlenecked, early-fault-tolerance gated.
- **Tensor-network baseline** (`qf-agent simulability`) — is the QAOA circuit classically simulable by an MPS? Entanglement entropy, bond dimension, MPS fidelity.

Methodology + citations: [docs/research-note-quantum-extras.md](#).

Surfaces

Surface	How
CLI	<code>qf-agent ...</code> — 11 commands (reference)
SDK	<code>from qf_agentos import load_spec, solve</code>
REST API	<code>qf-agent serve</code> — <code>sync /solve</code> , <code>async /jobs</code> (reference)
QF-Studio	web UI at / (bundled, no build step)
QF-Studio (React)	richer SPA in <code>studio-react/</code>
Docker	<code>docker compose up --build</code>

```
from qf_agentos import load_spec, solve

ctx = solve(load_spec("examples/collateral-allocation.yaml"))
print(ctx.state.audit.rendered)           # the FINAL DECISION block
print(ctx.state.bundle.manifest["evidence_digest"]) # deterministic per spec+seed
```

The agent team

1. **Requirements** — validate the Finance IR, discover missing constraints.
2. **Formulation** — enumerate LP / MILP / QUBO / Ising and what each can/cannot represent.
3. **Classical Baseline** — serious LP + MILP via HiGHS (never a strawman). 4/5. **Hardware Planner** — reduce to a qubit-sized instance, discover backends, estimate depth/cost, and decide whether quantum is warranted — **abstention is first-class**.

4. **Execution** — the instance ladder: MILP $\rightarrow$ QUBO-exact $\rightarrow$ SA $\rightarrow$ QAOA, under the policy engine.
5. **Verification** — objective recomputation, exact constraint checks, quantum-contribution accounting.
6. **Quantum-Advantage Auditor** — an honest outcome category + the FINAL DECISION block.
7. **Governance** — manifest, technical report, model card, deterministic evidence digest.

Outcome categories

CLASSICAL PREFERRED · QUANTUM NOT FEASIBLE ON PRESENT HARDWARE · QUANTUM PARITY · QUANTUM RESEARCH CANDIDATE · QUANTUM IMPROVEMENT OBSERVED · INDEPENDENT REPRODUCTION REQUIRED · POTENTIAL OPERATIONAL ADVANTAGE

The auditor’s bias is toward classical: a quantum result must be **verified feasible** and at least match the classical optimum before it earns anything better than “classical preferred”.

Human-control model (autonomy levels)

Level	Capability	Extra gate
L0	Explain and recommend	—
L1	Generate experiment plan	—
L2	Execute simulators automatically	—
L3	Execute paid QPU jobs	budget + human approval
L4	Recommend production decisions	human approval mandatory

The system never places trades, moves money, alters limits/fraud rules, runs paid QPU jobs without approval, or claims quantum advantage without verification.

What this can and cannot tell you

Honest limitations — read these.

- **Scale ceiling.** Quantum runs on *reduced research instances* ($\leq \sim 22$ qubits). At these sizes exact classical methods win trivially. This platform **cannot** tell you where a future crossover lies at production scale; it tells you the honest answer *today*, and gives you the instrument to detect a crossover when hardware arrives.
- **The verdict is mostly decided by the encoding, not the solver.** A QUBO often cannot faithfully represent a real financial constraint at tractable penalty weights (see the settlement finding in [FINDINGS](#)). This is a structural result, not a tuning failure.
- **“Agentic” means a deterministic agent pipeline** — no LLM reasoning at runtime. Same spec + seed $\Rightarrow$ byte-identical evidence digest.
- **This is a research instrument and methodology standard**, not a trading system.

Configuration

Environment variables (prefix QF_) — full table in [docs/CONFIGURATION.md](#): QF_LOG_LEVEL, QF_LOG_FORMAT, QF_DEFAULT_SEED, QF_STATEVECTOR_QUBIT_LIMIT, QF_EVIDENCE_DIR, QF_REGISTRY_BACKEND (file/mlflow/postgres), QF_API_KEYS, and credentials QF_IBM_TOKEN / QF_DWAVE_TOKEN (secrets, never logged).

Project layout

```
src/qf_agentos/
├─ core/           # Finance IR, typed state, artifacts, policy, config, logging, workflow
├─ finance/        # 5 domains + qml/qkernel, qae (amplitude estimation), tensor_network
├─ backends/       # protocol, registry, solvers (exact/SA/QAOA), IBM/D-Wave/PennyLane
├─ agents/         # the 9 agents (one module each)
├─ arena/          # QF-Arena benchmark runner + leaderboard
├─ governance/     # evidence bundle + file / MLflow / Postgres registries
├─ skills/         # Quantum Skills plugin registry (5 skills)
```

```
|— studio/      # bundled web UI (served at /)
|— api.py       # FastAPI service (sync + async jobs)
|— cli.py       # the `qf-agent` command (11 commands)
studio-react/  # richer React + TypeScript SPA (separate deployable)
docs/          # architecture, findings, CLI, API, configuration, families, research notes
examples/      # 7 runnable specs
tests/         # 229 tests, ~92% coverage, mypy --strict
```

Documentation

- [Findings](#) — the honest results, and *why* quantum loses
- [Problem families](#) · [CLI](#) · [REST API](#) · [Configuration](#)
- [Architecture](#) · [Real QPU runbook](#)
- [Research note: QAE + tensor networks](#)
- [Contributing & extension guide](#) · [Security & safety model](#) · [Changelog](#)

PDF (printable / shareable)

Typeset editions of everything above live in [docs/pdf/](#):

- [QF-AgentOS-Documentation.pdf](#) — the complete manual (cover, contents, every doc)
- [QF-AgentOS-Findings.pdf](#) — the results as a standalone report
- One PDF per document alongside them.

Rebuild after editing any Markdown (needs pandoc + a TeX distribution):

```
python3 scripts/build-pdfs.py
```

Disclaimer

Experimental research software. Not investment advice, not a licensed advisor, and not a production trading system. Outputs are decision-support only.

License

Apache-2.0. See [LICENSE](#) and [NOTICE](#).